MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY, WEST BENGAL

Paper Code: PCC-CSBS301/PCC-CS301 Data Structure & Algorithms

UPID: 003443

Time Allotted: 3 Hrs | Full Marks: 70 Marks

The Figures in the margin indicates full marks. Candidate are required to give their answer in their own words as far as Practicable

Group-A (Very Short Answer Type Question)

1. Answer any ten of the following :

i) What is the time complexity of an infix to postfix conversion algorithm?

Ans. The time complexity of an infix to postfix notation is O(n).

ii) Which of the following is essential for converting an infix expression to postfix notation?

Ans. Operator stack is used to convert infix expression to postfix form whereas operand stack is used to convert postfix to infix notation.

iii) A data structures a particular way of organisingand storing data either in computer's memory or on the disk storage so that iscan be used efficiently.

iv) Quick Sortis not a stable sorting algorithm?

v) Time complexity of bubble sort in best case isO(n)

vi) When a sorting technique is called stable?

Ans. Sorting Technique is called stable if it keeps elements with equal keys in the same relative order in the output as they were in the input.

vii) What is the search complexity in direct addressing?

Ans. O (1)

viii) What is the best case for linear search?

Ans. the best case for a linear search is when the target value is equal to the first element of the list.

ix) Reverse Polish notation is often known as postfix notation

x) The values in a BST can be sorted in ascending order by using which of the traversals method?

Ans. In-order traversal

xi) Which notation comprises a set of all functions h(n) that are greater than or equal to cg(n) for all values of n >= n0 ?

Ans. Big-Omega notation.

xii) A circular queue is implemented using an array of size 10. The array index starts with 0. front is 6. and rear is 9. The insertion of next element takes place at the array index.

Ans. 0

Group-B (Short Answer Type Question)

Answer any three of the following

2. Write an algorithm for binary search technique.

Ans. Input: A sorted array arr of size n, and a key x to be searched.

Output: Index of x in arr if x is present, else -1.

Algorithm:

Set start to 0 and end to n-1.

Repeat the following steps while start is less than or equal to end:

Calculate the middle index mid as (start + end) / 2.

If arr[mid] is equal to x, return mid.

If arr[mid] is less than x, set start to mid + 1.

If arr[mid] is greater than x, set end to mid - 1.

If the loop exits, return -1 (indicating x is not found in arr).

3. State the differences between stack and queue data structure.

Ans. **Definition:**

Stack: A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, where the last element added is the first one to be removed.

Queue: A queue is a linear data structure that follows the First In, First Out (FIFO) principle, where the first element added is the first one to be removed.

Operations:

Stack: Supports operations like push (to add an element), pop (to remove the top element), and peek (to view the top element without removing it).

Queue: Supports operations like enqueue (to add an element), dequeue (to remove the front element), and peek (to view the front element without removing it).

Order of Elements:

Stack: Elements are processed in a last in, first out (LIFO) order.

Queue: Elements are processed in a first in, first out (FIFO) order.

Implementation:

Stack: Can be implemented using arrays or linked lists.

Queue: Can also be implemented using arrays or linked lists.

Real-world Analogy:

Stack: Similar to a stack of plates, where you can only add or remove the top plate.

Queue: Similar to a queue of people waiting in line, where the person who joined the line first gets served first.

4. Convert the following infix expression into postfix expression using the stack data structure with detailed explanation:

A-(B / C +(D/E * F)/G) * H

Ans. The postfix expression for the given infix expression A-(B/C+(D/E∗F)/G)∗H is ABC/DE∗F/G+/H∗-.

Step 1: Initialize an empty stack and an empty string to store the postfix expression.

Step 2: Scan the infix expression from left to right.

Step 3: For each character in the infix expression, do the following:

If the character is an operand (A-Z or a-z), add it to the postfix expression string.

If the character is an operator:

If the stack is empty or the top of the stack contains an opening parenthesis, push the operator onto the stack.

If the precedence of the operator is greater than the precedence of the operator at the top of the stack, push the operator onto the stack.

If the precedence of the operator is less than or equal to the precedence of the operator at the top of the stack, pop operators from the stack and add them to the postfix expression string until either the stack is empty, an opening parenthesis is encountered, or the precedence condition is satisfied. Then, push the current operator onto the stack.

If the character is an opening parenthesis, push it onto the stack.

If the character is a closing parenthesis, pop operators from the stack and add them to the postfix expression string until an opening parenthesis is encountered. Discard the opening parenthesis.

Step 4: After scanning the entire infix expression, pop any remaining operators from the stack and add them to the postfix expression string.

Step 5: The resulting string is the postfix expression.

For the given infix expression A-(B/C+(D/E∗F)/G)∗H, the postfix expression is ABC/DE∗F/G+/H∗-.

5. Show how the following polynomial can be represented using a linked list.

7 ∗ 2y ∗ 2 - 4x ∗ 2y - 5xy ∗ 2 - 2

Ans. To represent the given polynomial 7 ∗ 2y ∗ 2 - 4x ∗ 2y - 5xy ∗ 2 - 2 using a linked list, we can create a linked list where each node represents a term in the

polynomial. Each node will contain three fields: coefficient, x exponent, and y exponent. Here's how the polynomial can be represented using a linked list:

7 * 2y * 2 - 4x * 2y - 5xy * 2 - 2

Linked List Representation:

Node 1: coefficient = 7, x_exponent = 0, y_exponent = 0

Node 2: coefficient = 2, x_exponent = 0, y_exponent = 1

Node 3: coefficient = 2, x_exponent = 1, y_exponent = 0

Node 4: coefficient = -4, x_exponent = 1, y_exponent = 1

Node 5: coefficient = -5, x_exponent = 1, y_exponent = 1

Node 6: coefficient = 2, x_exponent = 1, y_exponent = 1

Node 7: coefficient = -2, x_exponent = 0, y_exponent = 0

6. Wite difference between b tree and be tree.

Ans. The main differences between B-trees and B*-trees are:

Splitting and Merging:

In B-trees, splitting and merging occur on a node when it becomes full or empty, respectively.

In B*-trees, splitting and merging are more aggressive and can occur even if a node is not full or empty, leading to a shallower tree.

Node Occupancy:

B-trees have a lower threshold for node occupancy, which can lead to more frequent splits and merges.

B*-trees have a higher threshold for node occupancy, reducing the frequency of splits and merges and maintaining a more balanced tree.

Storage Efficiency:

B*-trees are more storage-efficient than B-trees because they have fewer internal nodes due to the more aggressive splitting and merging strategy.

Search Performance:

B∗-trees typically have better search performance than B-trees due to their shallower structure, resulting from fewer splits and merges.

Group-B (Long Answer Type Question)

Answer any three of the following

7. (a) Why circular queue is better than simple queue?

Ans. Circular queues offer several advantages over simple queues:

Efficient Use of Space: Circular queues make more efficient use of the allocated space since elements can wrap around to the beginning of the queue, utilizing space that would otherwise be wasted in a simple queue.

Faster Operations: In a circular queue, both insertion and deletion operations are typically faster compared to a simple queue. This is because there is no need to shift elements when the queue reaches its capacity or when elements are dequeued from the front.

Simpler Implementation: Implementing a circular queue is often simpler than implementing a simple queue with similar functionalities, especially when it comes to handling wraparound scenarios.

Better Performance: Circular queues can offer better performance for scenarios where the queue is expected to have a large number of operations, as the overhead of shifting elements is eliminated.

Overall, circular queues are a more efficient and practical choice for many applications compared to simple queues.

(b) Evaluate the postfix expression using stack: 331 ∗ 9 -

Ans. Initialize an empty stack.

Start scanning the postfix expression from left to right.

For each character in the expression:

If the character is an operand (a digit), push it onto the stack.

If the character is an operator (∗ or -), pop the top two elements from the stack, perform the operation, and push the result back onto the stack.

After scanning the entire expression, the final result will be the only element left in the stack

Start with an empty stack: Stack: []

Scan the expression from left to right:

3: Push onto stack: Stack: [3]

3: Push onto stack: Stack: [3, 3]

1: Push onto stack: Stack: [3, 3, 1]

*:

Pop 1: Stack: [3, 3]

Pop 3: Stack: [9]

Push 9: Stack: [9]

9: Push onto stack: Stack: [9, 9]

-:

Pop 9: Stack: [9]

Pop 9: Stack: []

Perform 9 - 9: Result = 0

Push 0: Stack: [0]

After evaluating the expression, the result is 0

(c) Write a program to convert infix expression to its equivalent postfix expression using stack.

Ans.
```
def precedence(op):
    if op == '+' or op == '-':
        return 1
    if op == '*' or op == '/':
        return 2
    return 0
def infix_to_postfix(expression):
    stack = []
```

```
    postfix = ""
    for char in expression:
        if char.isalnum():
            postfix += char
elif char == '(':
stack.append('(')
elif char == ')':
            while stack and stack[-1] != '(':
                postfix += stack.pop()
stack.pop()  # Remove '(' from stack
        else:
            while stack and precedence(char) <= precedence(stack[-1]):
                postfix += stack.pop()
stack.append(char)
    while stack:
        postfix += stack.pop()
    return postfix

# Example usage
infix_expr = "A-(B/C+(D/E*F)/G)*H"
postfix_expr = infix_to_postfix(infix_expr)
print("Infix Expression:", infix_expr)
print("Postfix Expression:", postfix_expr)
```

8. Write the advantages disadvantages and uses of a circular linked list.

Ans. Advantages of Circular Linked Lists:

Efficient Operations: Insertion and deletion operations at the beginning and end of a circular linked list can be performed in O(1) time complexity, similar to a regular linked list.

Traversal: Circular linked lists can be easily traversed starting from any node, unlike linear linked lists where traversal is only possible from the head node.

Space Efficiency: Circular linked lists can save memory compared to doubly linked lists because they do not require separate pointers for the head and tail nodes.

Implementation of Circular Buffer: Circular linked lists can be used to implement a circular buffer, which is a data structure that allows continuous writing and reading of data in a loop without having to resize the buffer.

Disadvantages of Circular Linked Lists:

Complexity: Circular linked lists can be more complex to implement and manage compared to linear linked lists, especially when dealing with operations that involve reordering or inserting nodes in specific positions.

Traversal Issues: If not implemented carefully, circular linked lists can lead to infinite loops during traversal if there are issues with the pointers or node linking.

Uses of Circular Linked Lists:

Round-Robin Scheduling: Circular linked lists are used in operating systems for round-robin scheduling algorithms, where processes are executed in a circular order.

Music Player Playlist: Circular linked lists can be used to implement a playlist in a music player, where songs are played in a loop, and the last song is followed by the first song in the playlist.

Resource Allocation: Circular linked lists can be used to manage resources in a system where resources are allocated and deallocated in a circular manner, ensuring fair distribution and efficient use of resources

9. (a) What are the best worst and average case time complexity of binary search algorithm?

Ans. The best, worst, and average case time complexities of the binary search algorithm are as follows:

Best Case: O(1)

The best case occurs when the target element is found at the middle of the array in the first comparison.

Worst Case: O(log n)

The worst case occurs when the target element is not present in the array, and the algorithm has to keep dividing the array in half until it reaches a single element.

Average Case: O(log n)

The average case also has a time complexity of O(log n), as the algorithm divides the array in half at each step, leading to a logarithmic time complexity.

(b) How linear search is differing from binary search?

Ans. Linear search and binary search are two different algorithms used to search for an element in a list. Here are the main differences between them:

Algorithm:

Linear Search: Involves iterating over each element in the list sequentially until the target element is found or the end of the list is reached.

Binary Search: Requires a sorted list and works by repeatedly dividing the list in half and narrowing down the search range based on whether the middle element is greater than or less than the target element.

Time Complexity:

Linear Search: O(n) in the worst-case scenario, where n is the number of elements in the list.

Binary Search: O(log n) in the worst-case scenario, where n is the number of elements in the list. Binary search is more efficient for large lists.

Requirement:

Linear Search: Does not require the list to be sorted.

Binary Search: Requires the list to be sorted in ascending or descending order.

Comparison:

Linear Search: Compares each element in the list with the target element until a match is found or the end of the list is reached.

Binary Search: Compares the target element with the middle element of the sorted list to determine which half of the list to search next.

Space Complexity:

Both linear search and binary search have a space complexity of O(1), as they do not require additional space proportional to the input size to perform the search.

(c) What are the preconditions for performing binary search in an array?

Ans. Before performing binary search on an array, the array must meet the following preconditions:

Sorted Array: The array must be sorted in ascending or descending order. Binary search relies on the sorted nature of the array to determine which half of the array to search in.

Unique Elements: While binary search can be adapted for arrays with duplicate elements, it is typically used on arrays where all elements are unique. This ensures that the search can accurately determine the position of the target element.

Random Access: Binary search requires the ability to access elements in the array in constant time (O(1)). This means that the array should support random access, such as arrays in most programming languages.

Size of Array: Binary search is most effective on large arrays. For very small arrays, the overhead of the binary search algorithm may outweigh the benefits, and a linear search may be more efficient.

(d) Write a C program for linear search in an array.

Ans. #include <stdio.h>

```
int linear Search(int arr[], int n, int target) {
    for (int i = 0; i< n; i++) {
        if (arr[i] == target) {
            return i; // Return the index of the target element if found
        }
    }
    return -1; // Return -1 if target element is not found
}

int main() {
    int arr[] = {2, 5, 7, 9, 12, 15, 17};
```

```
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 12;

    int result = linearSearch(arr, n, target);

    if (result != -1) {
printf("Element found at index %d\n", result);
    } else {
printf("Element not found\n");
    }

    return 0;
}
```

10 (a). What is a Stack ADT? Explain their operations.

Ans. A Stack is an Abstract Data Type (ADT) that represents a collection of elements with two main operations: push and pop. It follows the Last In, First Out (LIFO) principle, where the last element added to the stack is the first one to be removed.

Operations of a Stack:

Push: Adds an element to the top of the stack.

Pop: Removes and returns the top element of the stack.

Peek (or Top): Returns the top element of the stack without removing it is Empty: Checks if the stack is empty.Is Full: Checks if the stack is full (if it has a fixed size).

Size: Returns the number of elements in the stack.

Example of Stack Operations:

Consider an example of a stack of integers:

Initially, the stack is empty: Stack = []

Push 5: Stack = [5]

Push 10: Stack = [5, 10]

Push 15: Stack = [5, 10, 15]

Pop: Returns 15, Stack = [5, 10]

Peek: Returns 10, Stack = [5, 10]

Size: Returns 2

isEmpty: Returns false

Pop: Returns 10, Stack = [5]

Pop: Returns 5, Stack = []

isEmpty: Returns true

Stacks are commonly used in programming for tasks such as managing function calls (call stack), implementing undo mechanisms, and parsing expressions

(b) Write an algorithm to delete an item from a circular array of queue. Also find the complexity of the algorithm.

Ans. To delete an item from a circular array representing a queue, we can follow these steps:

Check if the queue is empty (front == -1).

If not empty, move the front pointer to the next element (front = (front + 1) % MAX_SIZE).

If front becomes equal to rear, set front and rear to -1 to indicate an empty queue.

Return the deleted element.

Here's the algorithm the algorithm:

```
Algorithm Delete(Queue, front, rear)
1. if front == -1
2. return "Queue is empty"
3. deletedElement = Queue[front]
4. if front == rear
5. front = rear = -1
```

```
6. else
7. front = (front + 1) % MAX_SIZE
8. return deletedElement
```

11. (a) Define algorithm.

(b) What are the measures of performance of an algorithm? Explain.

Ans. The performance of an algorithm can be measured using several criteria, including:

Time Complexity: This measures the amount of time an algorithm takes to run as a function of the size of the input. It is typically expressed using Big O notation (e.g., O(n), O(log n), O(n^2)).

Space Complexity: This measures the amount of memory an algorithm requires to run as a function of the size of the input. It is also expressed using Big O notation.

Best Case, Worst Case, and Average Case: These describe the time or space complexity of an algorithm under different scenarios. The best case is the scenario in which the algorithm performs the fewest operations, the worst case is the scenario in which it performs the most operations, and the average case is the scenario that is expected to occur most often.

Optimality: An algorithm is said to be optimal if it achieves the best possible performance (e.g., the fewest operations or the least memory usage) for a given problem.

Scalability: This refers to how well an algorithm performs as the size of the input grows. An algorithm is said to be scalable if its performance does not significantly degrade as the input size increases.

Stability: In sorting algorithms, stability refers to whether the relative order of equal elements is preserved in the sorted output.

Correctness: This measures whether an algorithm produces the correct output for all possible inputs.

Simplicity and Understandability: These are subjective measures that assess how easy it is to understand and implement an algorithm.

(a) Define asymptotic notation and explain each types.

Ans. Asymptotic notation is used in computer science to describe the behaviour of a function as its input approaches infinity. It provides a concise way to analyse the performance of algorithms without getting bogged down in the details of exact runtimes or memory usage. The three main types of asymptotic notation are Big O, Big Omega, and Big Theta.

Big O Notation (O):

Big O notation describes the upper bound of an algorithm's running time or space complexity. It represents the worst-case scenario.

Example: If an algorithm has a time complexity of O(n), it means that the algorithm's running time grows linearly with the size of the input.

Big Omega Notation (Ω):

Big Omega notation describes the lower bound of an algorithm's running time or space complexity. It represents the best-case scenario.

Example: If an algorithm has a time complexity of Ω(n), it means that the algorithm's running time is at least linear with the size of the input.

Big Theta Notation (Θ):

Big Theta notation describes both the upper and lower bounds of an algorithm's running time or space complexity. It represents the average-case scenario.

Example: If an algorithm has a time complexity of Θ(n), it means that the algorithm's running time grows linearly with the size of the input, and this growth is both the best and worst case